

SC102 L14: 对话系统——详细扩充学习笔记

涵盖：ScriptableObject | 简易对话系统架构 | 对话节点类型 | UI 动画与交互 | 进阶对话系统与节点编辑器 | SKDialoguePlayer 分析

1 对话系统概述

[p.1-p.3]

对话系统 (Dialogue System) 是游戏开发中不可或缺的核心玩法模块之一。在 RPG (角色扮演游戏)、AVG (冒险游戏)、视觉小说等类型中，对话系统承担着推进剧情、传递信息、提供玩家选择的重要角色。

在第四单元“玩法系统编程”中，我们将之前学过的 C# 进阶知识、Unity 引擎 API、UI 系统等内容融会贯通，搭建游戏中常见的玩法模块。L14 正是围绕对话系统展开。

本课内容大纲

[p.2]

1. **ScriptableObject**: 理解 Unity 中数据持久化的核心机制，掌握创建与使用 ScriptableObject 的方法
2. **简易对话系统架构**: 从零搭建一个可用的对话系统，理解对话管理器的设计
3. **编写各类对话节点**: 学习不同类型的对话节点 (普通对话节点、选择节点、事件节点等)
4. **构建 UI 动画与互动**: 为对话系统添加打字机效果、角色头像切换、选项高亮等 UI 表现
5. **进阶对话系统与节点编辑器**: 设计可扩展的对话节点架构，实现可视化节点编辑
6. **SKDialoguePlayer 分析**: 阅读分析已有的成熟对话系统实现，学习架构设计思路

为什么需要对话系统架构设计？

一个良好设计的对话系统应该具备以下特征：

- **数据与表现分离**: 对话内容 (谁说、说什么、选项是什么) 与对话的表现 (UI 如何显示、动画如何播放) 分离

- **可扩展性**: 新增对话节点类型（如分支、条件判断、触发事件）时无需修改核心逻辑
- **编辑器友好**: 策划人员可以在 Inspector 或专门的节点编辑器中方便地编辑对话内容，无需修改代码
- **可复用性**: 一段对话可在多个场景、多个 NPC 间复用

2 ScriptableObject 详解

[p.5]

2.1 什么是 ScriptableObject?

ScriptableObject（可脚本化对象）是 Unity 提供的一种数据容器类，允许我们永久性地存储数据。与普通的 C# 类实例不同，ScriptableObject 实例可以作为 .asset 文件保存在项目中，数据和引用在编辑器重新编译或退出播放模式后依然保留。

2.2 ScriptableObject 与 MonoBehaviour 的对比

特性	MonoBehaviour	ScriptableObject
继承自	MonoBehaviour	ScriptableObject
挂载方式	挂载到 GameObject 上	作为.asset 文件存在
生命周期	Update/Start/Awake 等	无生命周期回调
场景相关性	与场景绑定	独立于场景
数据持久化	仅在运行时	编辑器内外均持久
内存占用	每个实例独立	多引用共享同一实例
序列化支持	公有字段/[SerializeField]	同 MonoBehaviour
主要用途	行为逻辑	数据配置

2.3 创建 ScriptableObject

2.3.1 第一步：定义 ScriptableObject 类

创建一个继承自 ScriptableObject 的类,并在类上方添加 [CreateAssetMenu] 特性以在 Unity 编辑器中快捷创建:

```
using UnityEngine;

[CreateAssetMenu(fileName = "NewDialogue", menuName = "Dialogue/Dialogue Data"
)]
public class DialogueData : ScriptableObject
{
    [TextArea(3, 10)]
    public string[] dialogueLines;    // 对话行数组
}
```

```
public string speakerName;           // 说话者名字
public Sprite speakerPortrait;       // 说话者头像
}
```

CreateAssetMenu 参数说明:

- **fileName**: 新建资产时的默认文件名
- **menuName**: 在 Assets/Create 菜单中的路径
- 还可以添加 **order** 参数控制菜单中的排序

2.3.2 第二步: 在编辑器中创建资产

在 Unity 编辑器中, 右键 Project 窗口 → Create → Dialogue → Dialogue Data, 即可生成一个 .asset 文件。该文件可以像其他资源一样被拖拽引用、打包进 AssetBundle。

2.3.3 第三步: 在代码中使用

```
public class DialogueTrigger : MonoBehaviour
{
    public DialogueData dialogue;    // 在 Inspector 中拖入 .asset 文件

    void Start()
    {
        // 直接使用 ScriptableObject 中的数据
        foreach (string line in dialogue.dialogueLines)
        {
            Debug.Log($"{dialogue.speakerName}: {line}");
        }
    }
}
```

2.4 ScriptableObject 的核心优势

1. **数据驱动设计 (Data-Driven Design)**: 将配置数据从代码中剥离, 策划可以在编辑器中直接修改对话内容, 无需改动代码重新编译。这是现代游戏开发的核心思想。
2. **内存高效 (Memory Efficient)**: 当多个 GameObject 引用同一个 ScriptableObject 资产时, 内存中只存在一份数据副本。例如 100 个 NPC 都引用同一段“欢迎对话”, 不会造成内存浪费。
3. **资产管线集成 (Asset Pipeline)**: ScriptableObject 是原生 Unity 资产, 支持版本控制 (Git/PlasticSCM)、AssetBundle 打包、Addressables 系统。
4. **编辑器可预览**: 在 Inspector 中可以查看和编辑 ScriptableObject 的内容, 所见即所得。

5. 解耦与模块化：不同系统之间通过 ScriptableObject 传递配置数据，降低耦合度。

2.5 ScriptableObject 的进阶用法

2.5.1 作为事件通道 (Event Channel)

ScriptableObject 可以承载 UnityEvent，实现系统间的松耦合通信：

```
[CreateAssetMenu(menuName = "Events/Dialogue Event Channel")]
public class DialogueEventChannel : ScriptableObject
{
    public UnityEvent<string> onDialogueLineDisplayed;
    public UnityEvent onDialogueEnded;

    public void RaiseLineDisplayed(string line)
    {
        onDialogueLineDisplayed?.Invoke(line);
    }

    public void RaiseDialogueEnded()
    {
        onDialogueEnded?.Invoke();
    }
}
```

2.5.2 作为枚举替代 (Enums Replacement)

用 ScriptableObject 替代传统枚举，使数据更加灵活、可在不修改代码的情况下增加新类型：

```
public abstract class DialogueNodeType : ScriptableObject
{
    public abstract void Execute(DialogueContext context);
}
```

2.5.3 运行时可变数据

虽然 ScriptableObject 主要用于静态配置，但也可以在运行时修改其字段值（注意：在编辑器中修改会持久化到.asset 文件，在运行时修改仅在当次运行有效，停止播放后恢复）。

```
// 运行时修改不会持久化到.asset文件
dialogueData.speakerName = "新名字"; // 仅在本次Play Mode中生效
```

2.6 ScriptableObject 在对话系统中的角色

在对话系统中，ScriptableObject 主要用于：

- 存储对话数据：对话行、角色信息、选项内容
- 定义对话节点：每个对话节点是一个 ScriptableObject，包含该节点的类型、文字、后续节点引用
- 配置对话 UI 样式：字体、颜色、打字速度、头像位置等
- 事件触发：通过 UnityEvent 在特定对话节点触发游戏逻辑（如给物品、开门、战斗）

3 简易对话系统架构

[p.4]

3.1 系统设计原则

一个简易对话系统的核心设计遵循单一职责原则（Single Responsibility Principle）：

- **DialogueData** (ScriptableObject)：纯数据，存储对话内容
- **DialogueManager** (MonoBehaviour 单例)：控制对话流程，管理对话状态
- **DialogueUI** (MonoBehaviour)：负责对话的视觉呈现，包括文字显示、头像切换、选项面板
- **DialogueTrigger** (MonoBehaviour)：挂载在 NPC 或触发器上，用于启动对话

3.2 核心组件详解

3.2.1 DialogueManager ——对话流程控制器

DialogueManager 是整个对话系统的核心，负责管理对话的播放、暂停、推进、结束等状态。通常使用单例模式实现全局访问。

```
public class DialogueManager : MonoBehaviour
{
    public static DialogueManager Instance { get; private set; }

    [SerializeField] private DialogueUI dialogueUI;

    private DialogueData currentDialogue;
    private int currentLineIndex;
    private bool isDialogueActive;

    private void Awake()
    {
        if (Instance == null)
            Instance = this;
        else
```

```
        Destroy(gameObject);
    }

    // 开始一段对话
    public void StartDialogue(DialogueData dialogue)
    {
        if (isDialogueActive) return;

        currentDialogue = dialogue;
        currentLineIndex = 0;
        isDialogueActive = true;

        dialogueUI.Show();
        DisplayNextLine();
    }

    // 显示下一行对话
    public void DisplayNextLine()
    {
        if (currentLineIndex >= currentDialogue.dialogueLines.Length)
        {
            EndDialogue();
            return;
        }

        string line = currentDialogue.dialogueLines[currentLineIndex];
        dialogueUI.DisplayLine(currentDialogue.speakerName, line);
        currentLineIndex++;
    }

    // 结束对话
    private void EndDialogue()
    {
        isDialogueActive = false;
        currentDialogue = null;
        dialogueUI.Hide();
    }

    // 玩家点击或按键推进对话
    private void Update()
    {
        if (isDialogueActive && Input.GetKeyDown(KeyCode.Space))
        {
            DisplayNextLine();
        }
    }
}
```

```
}
```

关键设计点：

- 状态管理：使用 `isDialogueActive` 标志位防止重复触发
- 索引游标： `currentLineIndex` 记录当前播放到第几行
- 边界检查：到达最后一行时自动结束对话
- UI 分离： `DialogueManager` 不直接操作 UI 元素，而是通过 `DialogueUI` 接口

3.2.2 DialogueUI ——对话视觉呈现

`DialogueUI` 负责将对话数据展示在屏幕上，通常包含：

- 对话面板 (**Dialogue Panel**)：背景框
- 角色名文本 (**Speaker Name Text**)：显示当前说话者名字
- 对话文本 (**Dialogue Text**)：显示当前对话内容
- 角色头像 (**Speaker Portrait**)：当前说话者的头像图片
- 继续提示 (**Continue Indicator**)：提示玩家点击继续的图标

```
public class DialogueUI : MonoBehaviour
{
    [Header("UI References")]
    [SerializeField] private GameObject dialoguePanel;
    [SerializeField] private Text speakerNameText;
    [SerializeField] private Text dialogueText;
    [SerializeField] private Image speakerPortrait;

    [Header("Animation")]
    [SerializeField] private float textSpeed = 0.05f;    // 打字速度

    private Coroutine typingCoroutine;

    public void Show()
    {
        dialoguePanel.SetActive(true);
    }

    public void Hide()
    {
        dialoguePanel.SetActive(false);
        speakerNameText.text = "";
    }
}
```

```
        dialogueText.text = "";
    }

    public void DisplayLine(string speaker, string line)
    {
        speakerNameText.text = speaker;

        // 启动打字机效果
        if (typingCoroutine != null)
            StopCoroutine(typingCoroutine);
        typingCoroutine = StartCoroutine(TypeText(line));
    }

    private IEnumerator TypeText(string line)
    {
        dialogueText.text = "";
        foreach (char c in line)
        {
            dialogueText.text += c;
            yield return new WaitForSeconds(textSpeed);
        }
    }
}
```

3.2.3 DialogueTrigger ——对话启动器

DialogueTrigger 挂载在场景中的 NPC 或触发器 GameObject 上, 当玩家与之交互时启动对话:

```
public class DialogueTrigger : MonoBehaviour
{
    [SerializeField] private DialogueData dialogue;
    [SerializeField] private KeyCode interactKey = KeyCode.E;

    private bool playerInRange;

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            playerInRange = true;
            // 显示交互提示 UI (如 "按 E 对话")
        }
    }

    private void OnTriggerExit(Collider other)
    {

```



```

        if (other.CompareTag("Player"))
        {
            playerInRange = false;
            // 隐藏交互提示 UI
        }
    }

    private void Update()
    {
        if (playerInRange && Input.GetKeyDown(interactKey))
        {
            DialogueManager.Instance.StartDialogue(dialogue);
        }
    }
}

```

3.3 简易对话系统整体数据流

1. 玩家进入 NPC 触发器范围 → DialogueTrigger.playerInRange = true
2. 玩家按下交互键 → DialogueTrigger 调用 DialogueManager.StartDialogue(dialogue)
3. DialogueManager 获取 DialogueData(ScriptableObject), 初始化状态, 调用 DialogueUI.Show()
4. DialogueManager.DisplayNextLine() 提取当前行数据, 传给 DialogueUI.DisplayLine()
5. DialogueUI 播放打字机动画, 显示说话者和内容
6. 玩家按空格/Space → DialogueManager.DisplayNextLine() 继续
7. 所有行播放完毕 → DialogueManager.EndDialogue() → DialogueUI.Hide()

4 各类对话节点设计

[p.2-p.3]

在简易对话系统的基础上, 为了支持更丰富的叙事需求, 我们需要不同类型的对话节点 (Dialogue Node)。

4.1 对话节点的抽象设计

使用面向对象多态设计节点体系, 所有节点继承自统一的基类:

```

public abstract class DialogueNode : ScriptableObject
{
    [TextArea] public string nodeTitle;           // 节点标题 (编辑器中辨识用)
}

```

```
// 节点执行入口，返回下一个应执行的节点 (null表示结束)
public abstract DialogueNode Execute(DialogueContext context);
}
```

DialogueContext 是一个上下文对象，携带着对话系统的运行时状态：

```
[System.Serializable]
public class DialogueContext
{
    public DialogueUI dialogueUI;
    public GameObject player;
    public Dictionary<string, bool> flags; // 用于条件判断的全局标记
    // ... 其他运行时数据
}
```

4.2 基础对话节点 (DialogueLineNode)

[p.5]

最基本、最常用的节点类型，用于显示一段角色对话。

```
[CreateAssetMenu(menuName = "Dialogue/Nodes/Line Node")]
public class DialogueLineNode : DialogueNode
{
    public string speakerName; // 说话者
    public Sprite speakerPortrait; // 说话者头像 (可选)
    [TextArea(3, 10)] public string text; // 对话内容
    public DialogueNode nextNode; // 下一节点

    public override DialogueNode Execute(DialogueContext context)
    {
        context.dialogueUI.DisplayLine(speakerName, text, speakerPortrait);
        return nextNode; // 返回下一个节点，等待玩家确认后继续
    }
}
```

4.3 选择节点 (DialogueChoiceNode)

允许玩家从多个选项中做出选择，不同选项导向不同的后续对话。

```
[System.Serializable]
public class DialogueChoice
{
    public string choiceText; // 选项文字
    public DialogueNode nextNode; // 此选项对应的后续节点
}
```

```
[CreateAssetMenu(menuName = "Dialogue/Nodes/Choice Node")]
public class DialogueChoiceNode : DialogueNode
{
    [TextArea(3, 5)] public string prompt;          // 提示文字 (如 "你打算怎么做?"
    )

    public List<DialogueChoice> choices;            // 选项列表

    public override DialogueNode Execute(DialogueContext context)
    {
        // 显示提示文字和选项
        context.dialogueUI.ShowChoices(prompt, choices);

        // 注意: 选择节点不立即返回 nextNode
        // 而是等待玩家选择后, 由 DialogueManager 调用对应的 choice.nextNode
        return null; // 等待中
    }

    // 当玩家做出选择后, DialogueManager 调用此方法
    public DialogueNode OnChoiceSelected(int choiceIndex)
    {
        if (choiceIndex >= 0 && choiceIndex < choices.Count)
            return choices[choiceIndex].nextNode;
        return null;
    }
}
```

4.4 分支节点 (DialogueBranchNode)

根据游戏状态 (如玩家等级、任务进度、是否有某物品) 自动选择不同的对话分支。

```
[System.Serializable]
public class DialogueBranch
{
    public string conditionKey;          // 条件键名 (如 "hasSword", "level>5"
    )

    public bool expectedValue = true;    // 期望值
    public DialogueNode nextNode;        // 满足条件时的后续节点
}

[CreateAssetMenu(menuName = "Dialogue/Nodes/Branch Node")]
public class DialogueBranchNode : DialogueNode
{
    public List<DialogueBranch> branches; // 按顺序检查的分支列表
    public DialogueNode fallbackNode;     // 都不满足时的默认节点

    public override DialogueNode Execute(DialogueContext context)
```

```

{
    foreach (var branch in branches)
    {
        if (context.flags.TryGetValue(branch.conditionKey, out bool value)
        )
        {
            if (value == branch.expectedValue)
                return branch.nextNode;
        }
    }
    return fallbackNode; // 默认分支
}
}

```

4.5 事件节点 (DialogueEventNode)

在对话中触发游戏事件，如给予物品、开启战斗、播放动画、改变好感度等。

```

[CreateAssetMenu(menuName = "Dialogue/Nodes/Event Node")]
public class DialogueEventNode : DialogueNode
{
    public UnityEvent onExecute; // 触发的事件
    public DialogueNode nextNode; // 事件后继续的节点

    public override DialogueNode Execute(DialogueContext context)
    {
        onExecute?.Invoke();
        return nextNode; // 立即返回下一节点（事件节点通常不阻塞对话）
    }
}

```

4.6 DialogueManager 对节点系统的适配

引入节点系统后，DialogueManager 需要修改为以节点为驱动：

```

public class DialogueManager : MonoBehaviour
{
    public static DialogueManager Instance { get; private set; }

    [SerializeField] private DialogueUI dialogueUI;

    private DialogueNode currentNode;
    private bool isDialogueActive;
    private DialogueContext context;

    private void Awake()
    {

```

```
{
    if (Instance == null) Instance = this;
    else Destroy(gameObject);
}

public void StartDialogue(DialogueNode startNode)
{
    if (isDialogueActive) return;

    isDialogueActive = true;
    context = new DialogueContext
    {
        dialogueUI = dialogueUI,
        flags = new Dictionary<string, bool>()
    };

    dialogueUI.Show();
    ExecuteNode(startNode);
}

private void ExecuteNode(DialogueNode node)
{
    if (node == null)
    {
        EndDialogue();
        return;
    }

    currentNode = node;
    DialogueNode nextNode = node.Execute(context);

    // 如果节点返回了下一个节点且不是选择节点，继续执行
    // （选择节点返回 null，等待玩家选择后再推进）
    if (nextNode != null)
    {
        ExecuteNode(nextNode);
    }
}

// 玩家点击继续（用于 LineNode 后推进）
public void AdvanceDialogue()
{
    if (currentNode is DialogueLineNode lineNode)
    {
        ExecuteNode(lineNode.nextNode);
    }
}
```

```
}

// 玩家做出选择
public void SelectChoice(int choiceIndex)
{
    if (currentNode is DialogueChoiceNode choiceNode)
    {
        DialogueNode next = choiceNode.OnChoiceSelected(choiceIndex);
        ExecuteNode(next);
    }
}

public void SetFlag(string key, bool value)
{
    context.flags[key] = value;
}

private void EndDialogue()
{
    isDialogueActive = false;
    currentNode = null;
    dialogueUI.Hide();
}
}
```

4.7 节点设计模式总结

对话节点的设计本质上是**策略模式（Strategy Pattern）**和**责任链模式（Chain of Responsibility）**的结合：

- 每个节点类型是一个**策略**，封装了该节点的特定行为（显示文字、展示选项、判断分支等）
- 节点通过 `nextNode` 引用形成**责任链**，对话沿着节点链向前推进
- 使用 `ScriptableObject` 作为节点载体，使得整个对话树可以在编辑器中以可视化方式搭建

5 构建 UI 动画与互动

[p.2]

对话系统不仅要功能完善，还需要良好的视觉呈现。本节详细讲解对话 UI 中的常见动画和交互效果实现。

5.1 打字机效果（Typewriter Effect）

打字机效果是对话系统中最重要动画之一。文字逐个显示，模拟角色正在说话的感觉。

```
public class TypewriterEffect : MonoBehaviour
{
    [SerializeField] private Text dialogueText;
    [SerializeField] private float defaultSpeed = 0.05f;    // 默认每个字的间隔
    [SerializeField] private float punctuationDelay = 0.2f; // 标点符号额外延迟

    private Coroutine typingCoroutine;
    private string fullText;
    private bool isTyping;

    // 开始打字
    public void StartTyping(string text)
    {
        fullText = text;

        if (typingCoroutine != null)
            StopCoroutine(typingCoroutine);

        typingCoroutine = StartCoroutine(TypeText());
    }

    private IEnumerator TypeText()
    {
        isTyping = true;
        dialogueText.text = "";

        for (int i = 0; i < fullText.Length; i++)
        {
            dialogueText.text += fullText[i];

            // 标点符号需要更长的暂停, 增加"说话感"
            float delay = defaultSpeed;
            if (fullText[i] == '.' || fullText[i] == '!' || fullText[i] == '?')
            {
                delay += punctuationDelay;
            }
            else if (fullText[i] == ',')
            {
                delay += punctuationDelay * 0.5f;
            }

            yield return new WaitForSeconds(delay);
        }

        isTyping = false;
    }
}
```

```
// 立即完成打字（玩家点击跳过）
public void SkipTyping()
{
    if (typingCoroutine != null)
        StopCoroutine(typingCoroutine);
    dialogueText.text = fullText;
    isTyping = false;
}

public bool IsTyping => isTyping;
}
```

交互逻辑增强：当玩家点击时，如果打字动画正在进行中，则立即完成当前行；如果已完成当前行，则推进到下一行。这提供了流畅的交互体验：

```
public void OnPlayerClick()
{
    if (typewriterEffect.IsTyping)
    {
        typewriterEffect.SkipTyping();
    }
    else
    {
        DialogueManager.Instance.AdvanceDialogue();
    }
}
```

5.2 对话面板动画

对话面板的显示和隐藏应配合平滑的动画，提升整体品质感。

```
public class DialoguePanelAnimator : MonoBehaviour
{
    [SerializeField] private RectTransform panelRect;
    [SerializeField] private float animDuration = 0.3f;
    [SerializeField] private AnimationCurve easeCurve = AnimationCurve.EaseInOut(0, 0, 1, 1);

    private Coroutine animCoroutine;
    private Vector2 hiddenPos; // 隐藏时的位置（屏幕下方外）
    private Vector2 shownPos; // 显示时的位置

    private void Start()
    {
        shownPos = panelRect.anchoredPosition;
        hiddenPos = shownPos - new Vector2(0, panelRect.rect.height + 50);
    }
}
```



```

        panelRect.anchoredPosition = hiddenPos; // 初始隐藏
    }

    public void Show()
    {
        if (animCoroutine != null) StopCoroutine(animCoroutine);
        animCoroutine = StartCoroutine(AnimateTo(shownPos));
    }

    public void Hide()
    {
        if (animCoroutine != null) StopCoroutine(animCoroutine);
        animCoroutine = StartCoroutine(AnimateTo(hiddenPos));
    }

    private IEnumerator AnimateTo(Vector2 targetPos)
    {
        Vector2 startPos = panelRect.anchoredPosition;
        float elapsed = 0f;

        while (elapsed < animDuration)
        {
            elapsed += Time.deltaTime;
            float t = elapsed / animDuration;
            float easedT = easeCurve.Evaluate(t);
            panelRect.anchoredPosition = Vector2.Lerp(startPos, targetPos,
                easedT);
            yield return null;
        }

        panelRect.anchoredPosition = targetPos;
    }
}

```

5.3 角色头像切换动画

当说话者切换时，头像应有过渡效果（如淡入淡出、轻微缩放）：

```

public class PortraitDisplay : MonoBehaviour
{
    [SerializeField] private Image portraitImage;
    [SerializeField] private float crossfadeDuration = 0.2f;

    private Coroutine fadeCoroutine;

    public void SetPortrait(Sprite newPortrait)
    {
    }
}

```

```
{
    if (fadeCoroutine != null) StopCoroutine(fadeCoroutine);
    fadeCoroutine = StartCoroutine(CrossfadePortrait(newPortrait));
}

private IEnumerator CrossfadePortrait(Sprite target)
{
    // 如果当前没有头像，直接设置
    if (portraitImage.sprite == null)
    {
        portraitImage.sprite = target;
        portraitImage.color = Color.white;
        yield break;
    }

    // 淡出
    float elapsed = 0f;
    while (elapsed < crossfadeDuration * 0.5f)
    {
        elapsed += Time.deltaTime;
        float t = elapsed / (crossfadeDuration * 0.5f);
        portraitImage.color = Color.Lerp(Color.white, Color.clear, t);
        yield return null;
    }

    // 切换图片
    portraitImage.sprite = target;

    // 淡入
    elapsed = 0f;
    while (elapsed < crossfadeDuration * 0.5f)
    {
        elapsed += Time.deltaTime;
        float t = elapsed / (crossfadeDuration * 0.5f);
        portraitImage.color = Color.Lerp(Color.clear, Color.white, t);
        yield return null;
    }

    portraitImage.color = Color.white;
}
}
```

5.4 选项按钮交互

选项面板应具有清晰的视觉反馈（悬停高亮、点击动效）：

```
public class ChoiceButton : MonoBehaviour, IPointerEnterHandler,
    IPointerExitHandler, IPointerClickHandler
{
    [SerializeField] private Text choiceText;
    [SerializeField] private Image background;
    [SerializeField] private Color normalColor = Color.white;
    [SerializeField] private Color hoverColor = Color.yellow;
    [SerializeField] private float hoverScale = 1.05f;

    private int choiceIndex;
    private System.Action<int> onSelected;

    public void Setup(string text, int index, System.Action<int> callback)
    {
        choiceText.text = text;
        choiceIndex = index;
        onSelected = callback;
        background.color = normalColor;
        transform.localScale = Vector3.one;
    }

    public void OnPointerEnter(PointerEventData eventData)
    {
        background.color = hoverColor;
        transform.localScale = Vector3.one * hoverScale;
    }

    public void OnPointerExit(PointerEventData eventData)
    {
        background.color = normalColor;
        transform.localScale = Vector3.one;
    }

    public void OnPointerClick(PointerEventData eventData)
    {
        // 点击时播放一个小动画
        StartCoroutine(ClickAnimation());
        onSelected?.Invoke(choiceIndex);
    }

    private IEnumerator ClickAnimation()
    {
        // 点击缩放弹跳效果
        transform.localScale = Vector3.one * 0.9f;
        yield return new WaitForSeconds(0.05f);
    }
}
```

```

        transform.localScale = Vector3.one * 1.1f;
        yield return new WaitForSeconds(0.05f);
        transform.localScale = Vector3.one;
    }
}

```

6 进阶对话系统与节点编辑器

[p.6]

6.1 进阶对话系统的设计目标

一个进阶的对话系统应满足以下要求：

- **支持更多功能：**分支对话、条件判断、事件触发、变量系统、本地化支持
- **拥有可视化的编辑器：**策划人员可以直观地设计对话树，无需编辑代码或手动拼接节点
- **使用简单：**添加对话节点的 API 应简洁明了，降低使用门槛
- **可扩展：**支持自定义节点类型，方便不同项目需求

6.2 节点编辑器（Node Editor）的核心概念

节点编辑器是一种可视化的图编辑工具。在 Unity 中，通常基于以下技术实现：

1. **Unity Editor GUI：**使用 `EditorWindow` + `Handles` + `GUI` 绘制自定义编辑器窗口
2. **图结构（Graph）：**节点（Node）作为图的顶点，连接线（Edge）表示节点间的转移关系
3. **序列化（Serialization）：**通过 `ScriptableObject` 保存整个对话图的结构数据

6.3 节点编辑器架构设计

6.3.1 数据模型

```

// 表示整个对话图
[CreateAssetMenu(menuName = "Dialogue/Dialogue Graph")]
public class DialogueGraph : ScriptableObject
{
    public List<DialogueNodeData> nodes = new List<DialogueNodeData>();
}

// 序列化友好的节点数据（非ScriptableObject，直接存储在图中）
[System.Serializable]
public class DialogueNodeData

```

```

{
    public string guid;                // 唯一标识符
    public Vector2 position;           // 在编辑器中的位置
    public string nodeType;            // 节点类型名称
    public string jsonData;            // 节点特有数据的JSON序列化
    public List<string> outputPorts = new List<string>(); // 输出端口
    public List<EdgeData> edges = new List<EdgeData>(); // 从此节点出发的
        连线
}

// 连线数据
[System.Serializable]
public class EdgeData
{
    public string sourcePortGuid;      // 源端口 GUID
    public string targetNodeGuid;     // 目标节点 GUID
}

```

6.3.2 编辑器窗口

```

public class DialogueGraphEditor : EditorWindow
{
    private DialogueGraph currentGraph;
    private Vector2 scrollPosition;
    private Vector2 graphOffset;

    // 节点样式
    private float nodeWidth = 200f;
    private float nodeHeight = 100f;

    // 选中和拖拽状态
    private DialogueNodeData selectedNode;
    private bool isDraggingNode;
    private Vector2 dragOffset;

    [MenuItem("Window/Dialogue Graph Editor")]
    public static void OpenWindow()
    {
        var window = GetWindow<DialogueGraphEditor>();
        window.titleContent = new GUIContent("Dialogue Graph");
    }

    private void OnGUI()
    {
        // 绘制工具栏
    }
}

```

```

    DrawToolbar();

    // 绘制节点
    DrawNodes();

    // 绘制连线
    DrawEdges();

    // 处理输入事件
    ProcessEvents(Event.current);

    // 强制重绘以实现实时更新
    if (GUI.changed) Repaint();
}

private void DrawToolbar()
{
    EditorGUILayout.BeginHorizontal(EditorStyles.toolbar);
    currentGraph = (DialogueGraph)EditorGUILayout.ObjectField(
        currentGraph, typeof(DialogueGraph), false);

    if (GUILayout.Button("Add Line Node", EditorStyles.toolbarButton))
        AddNode<DialogueLineNode>("Line Node");
    if (GUILayout.Button("Add Choice Node", EditorStyles.toolbarButton))
        AddNode<DialogueChoiceNode>("Choice Node");
    if (GUILayout.Button("Add Branch Node", EditorStyles.toolbarButton))
        AddNode<DialogueBranchNode>("Branch Node");
    if (GUILayout.Button("Add Event Node", EditorStyles.toolbarButton))
        AddNode<DialogueEventNode>("Event Node");

    EditorGUILayout.EndHorizontal();
}

private void DrawNodes()
{
    if (currentGraph == null) return;

    // 开始滚动视图
    scrollPosition = GUI.BeginScrollView(
        new Rect(0, EditorStyles.toolbar.fixedHeight, position.width,
            position.height - EditorStyles.toolbar.fixedHeight),
        scrollPosition,
        new Rect(0, 0, 5000, 5000)); // 大画布

    // 绘制网格（可选，增强可视化）
    DrawGrid(5000, 5000, 20, Color.gray * 0.3f);
}

```

```

        foreach (var node in currentGraph.nodes)
        {
            DrawNode(node);
        }

        GUI.EndScrollView();
    }

    private void DrawNode(DialogueNodeData node)
    {
        Rect nodeRect = new Rect(
            node.position.x + graphOffset.x,
            node.position.y + graphOffset.y,
            nodeWidth, nodeHeight);

        // 绘制节点背景
        GUI.Box(nodeRect, "", EditorStyles.helpBox);

        // 绘制节点标题
        GUI.Label(new Rect(nodeRect.x + 5, nodeRect.y + 5, nodeWidth - 10, 20)
            ,
                node.nodeType, EditorStyles.boldLabel);

        // 绘制输入端口（左侧）
        GUI.Button(new Rect(nodeRect.x - 8, nodeRect.y + nodeHeight / 2 - 8,
            16, 16), "");

        // 绘制输出端口（右侧）
        GUI.Button(new Rect(nodeRect.x + nodeWidth - 8, nodeRect.y +
            nodeHeight / 2 - 8, 16, 16), "");
    }

    private void DrawEdges()
    {
        if (currentGraph == null) return;

        foreach (var node in currentGraph.nodes)
        {
            foreach (var edge in node.edges)
            {
                var targetNode = currentGraph.nodes.Find(n => n.guid == edge.targetNodeGuid);
                if (targetNode != null)
                {
                    Vector2 startPos = node.position + new Vector2(nodeWidth,

```

```

        nodeHeight / 2);
        Vector2 endPos = targetNode.position + new Vector2(0,
            nodeHeight / 2);

        // 绘制贝塞尔曲线连线
        Handles.DrawBezier(startPos, endPos,
            startPos + Vector2.right * 50,
            endPos + Vector2.left * 50,
            Color.white, null, 2f);
    }
}

private void DrawGrid(float width, float height, float gridSpacing, Color
    color)
{
    // 绘制网格线以提供视觉参考（简化实现）
}

private void ProcessEvents(Event e)
{
    // 处理右键菜单（添加节点）
    // 处理拖拽（移动节点）
    // 处理连线操作（从输出端口拖到输入端口）
    // 处理删除操作
}

private void AddNode<T>(string displayName) where T : DialogueNode
{
    if (currentGraph == null) return;

    var node = new DialogueNodeData
    {
        guid = System.Guid.NewGuid().ToString(),
        position = Vector2.zero - graphOffset,
        nodeType = displayName
    };

    currentGraph.nodes.Add(node);
    EditorUtility.SetDirty(currentGraph);
}
}

```


6.4 节点编辑器中的连线逻辑

连线操作（用户从节点 A 的输出端口拖拽到节点 B 的输入端口）需要：

1. 检测鼠标按下在输出端口上
2. 从该端口绘制临时拖拽线（跟随鼠标）
3. 检测鼠标释放在输入端口上
4. 创建/更新 EdgeData 记录
5. 标记 Graph 为 dirty 以触发序列化保存

6.5 运行时解析 DialogueGraph

在运行时，将 DialogueGraph 中的 DialogueNodeData 列表解析为可执行的节点对象：

```
public class DialogueGraphRunner
{
    private DialogueGraph graph;
    private Dictionary<string, DialogueNode> runtimeNodes;

    public void LoadGraph(DialogueGraph dialogueGraph)
    {
        graph = dialogueGraph;
        runtimeNodes = new Dictionary<string, DialogueNode>();

        // 将序列化数据转换为运行时节点
        foreach (var nodeData in graph.nodes)
        {
            DialogueNode runtimeNode = CreateRuntimeNode(nodeData);
            runtimeNodes[nodeData.guid] = runtimeNode;
        }

        // 连接节点关系
        foreach (var nodeData in graph.nodes)
        {
            var runtimeNode = runtimeNodes[nodeData.guid];
            foreach (var edge in nodeData.edges)
            {
                if (runtimeNodes.TryGetValue(edge.targetNodeGuid, out var
                    nextNode))
                {
                    // 根据节点类型设置 nextNode 引用
                    SetNextNode(runtimeNode, nextNode);
                }
            }
        }
    }
}
```

```

    }
}

private DialogueNode CreateRuntimeNode(DialogueNodeData data)
{
    // 从 jsonData 反序列化或工厂模式创建
}
}

```

7 SKDialoguePlayer 及相关组件分析

[p.6]

SKDialoguePlayer 是 Silverjay 课程提供的一个已经实现的进阶对话系统示例。通过分析其源码，可以学习到成熟对话系统的架构设计思路。

7.1 架构概览

SKDialoguePlayer 采用了 MVC（Model-View-Controller）风格的分层架构：

- **Model(数据层):** ScriptableObject 存储的对话节点数据 (DialogueNodeData, DialogueGraph)
- **View (视图层):** DialogueUI, 负责对话的视觉呈现、动画播放
- **Controller (控制层):** SKDialoguePlayer, 作为对话的核心驱动, 读取数据、控制流程、协调视图更新

7.2 核心设计模式分析

7.2.1 状态机模式 (State Machine)

SKDialoguePlayer 内部使用有限状态机 (FSM) 管理对话的不同阶段：

```

public enum DialogueState
{
    Idle,           // 空闲，无对话进行
    Playing,        // 正在播放对话（打字机动画进行中）
    WaitingInput,   // 等待玩家按键推进
    Choosing,       // 等待玩家选择选项
    Finished        // 对话结束
}

```

每个状态对应不同的行为：

- **Idle:** 可以接收新的 StartDialogue 请求

- **Playing:** 文字逐字显示，点击可跳过
- **WaitingInput:** 当前行已完成，等待点击推进到下一行
- **Choosing:** 选项面板可见，等待玩家点击选项
- **Finished:** 清理状态，隐藏 UI

7.2.2 观察者模式 (Observer Pattern)

SKDialoguePlayer 通过事件系统通知外部系统对话的状态变化，实现松耦合：

```
public class SKDialoguePlayer : MonoBehaviour
{
    // 对话事件
    public UnityEvent<DialogueNode> OnNodeEnter;           // 进入某节点
    public UnityEvent<DialogueNode> OnNodeExit;           // 离开某节点
    public UnityEvent<string> OnLineDisplayed;             // 显示某行文字
    public UnityEvent<int> OnChoiceSelected;              // 玩家选择了某个选项
    public UnityEvent OnDialogueStart;                    // 对话开始
    public UnityEvent OnDialogueEnd;                      // 对话结束

    // 这些事件可以被其他组件订阅，例如：
    // - 相机系统订阅 OnDialogueStart，锁定视角
    // - 音频系统订阅 OnLineDisplayed，播放角色语音
    // - 任务系统订阅 OnDialogueEnd，更新任务进度
}
```

7.2.3 命令模式 (Command Pattern)

每个对话节点的操作被封装为”命令”：

```
public interface IDialogueCommand
{
    IEnumerator Execute(DialogueContext context);
    bool IsComplete { get; }
}

// 具体命令示例
public class ShowTextCommand : IDialogueCommand
{
    public string text;
    public string speaker;

    public IEnumerator Execute(DialogueContext context)
    {
        yield return context.ui.TypeText(text, speaker);
    }
}
```

```

    }

    public bool IsComplete => !context.ui.IsTyping;
}

public class WaitInputCommand : IDialogueCommand
{
    public IEnumerator Execute(DialogueContext context)
    {
        yield return new WaitUntil(() => Input.GetKeyDown(KeyCode.Space));
    }

    public bool IsComplete => true;
}

```

这种设计的优势：

- 对话节点可以由多个子命令组合而成
- 新增命令类型不影响现有代码
- 命令可以链式执行，支持复杂的时间线编排

7.3 SKDialoguePlayer 的数据结构设计

SKDialoguePlayer 使用 ScriptableObject 定义对话树，核心数据结构如下：

```

// 对话资产（顶层容器）
public class SKDialogueAsset : ScriptableObject
{
    public List<SKNodeData> nodes;
    public string startNodeId;
}

// 节点数据（节点基类）
[System.Serializable]
public class SKNodeData
{
    public string id; // 节点唯一 ID
    public Vector2 editorPosition; // 编辑器位置
    public List<SKConnectionData> outputs; // 输出连接
}

// 对话行节点
[System.Serializable]
public class SKLineNodeData : SKNodeData
{

```

```

    public string characterName;
    public string dialogueText;
    public Sprite characterPortrait;
}

// 选择节点
[System.Serializable]
public class SKChoiceNodeData : SKNodeData
{
    public string promptText;
    public List<SKChoiceOption> options;
}

[System.Serializable]
public class SKChoiceOption
{
    public string text;
    public string targetNodeId;
}

// 连接数据
[System.Serializable]
public class SKConnectionData
{
    public string portName;          // 端口名（一个节点可以有多个输出端口）
    public string targetNodeId;      // 目标节点ID
}

```

7.4 SKDialoguePlayer 与 UI 的协作

SKDialoguePlayer 不直接依赖具体的 UI 实现类，而是通过接口（Interface）定义契约：

```

public interface IDialogueView
{
    void Show();
    void Hide();
    void DisplayDialogueLine(string speaker, string text, Sprite portrait);
    IEnumerator ShowChoices(string prompt, List<ChoiceData> choices);
    void SetContinueIndicator(bool visible);
    event System.Action OnContinueClicked;
    event System.Action<int> OnChoiceSelected;
}

```

这种设计的好处：

- SKDialoguePlayer 可以配合任意实现了 IDialogueView 的 UI 系统

- 方便进行单元测试（使用 Mock IDialogueView）
- UI 实现可以被替换而无需修改对话逻辑

7.5 SKDialoguePlayer 对话生命周期

1. 初始化 (**Initialize**): 加载 SKDialogueAsset, 构建运行时节点字典, 注册事件监听
2. 开始 (**Start**): 定位到 startNodeId, 发出 OnDialogueStart 事件, 进入 Playing 状态
3. 推进行 (**Advance**): 执行当前节点的命令序列, 逐字显示文字
4. 选择 (**Choice**): 如果是 ChoiceNode, 进入 Choosing 状态, 等待玩家选择
5. 跳转 (**Transition**): 根据连接数据跳转到下一个节点
6. 结束 (**End**): 到达终点节点 (没有输出连接), 发出 OnDialogueEnd 事件, 回到 Idle 状态

8 实践练习与代码分析

[p.4-p.6]

8.1 练习一: 创建并使用 ScriptableObject

要求: 创建一个继承自 ScriptableObject 的对话数据类, 包含说话者名字、对话行数组、说话者头像。然后在 Unity 编辑器中创建一个资产实例, 并在代码中读取和使用。

解题思路:

1. 创建 C# 脚本, 定义继承自 ScriptableObject 的类
2. 使用 [CreateAssetMenu] 特性使其可在编辑器中创建
3. 定义需要的公共字段: `string speakerName`、`string[] dialogueLines`、`Sprite speakerPortrait`
4. 在编辑器中: 右键 → Create → 对应菜单 → 填写数据
5. 在 MonoBehaviour 中声明 `public DialogueData dialogue`, 在 Inspector 中拖入资产
6. 在代码中通过 `dialogue.speakerName`、`dialogue.dialogueLines` 访问数据

完整代码:

```
// DialogueData.cs —— ScriptableObject 定义
using UnityEngine;

[CreateAssetMenu(fileName = "NewDialogue", menuName = "Dialogue/Simple
    Dialogue")]
public class DialogueData : ScriptableObject
```

```
{  
    public string speakerName;  
    [TextArea(3, 10)]  
    public string[] dialogueLines;  
    public Sprite speakerPortrait;  
}
```

```
// DialogueTester.cs —— 使用 ScriptableObject  
using UnityEngine;  
  
public class DialogueTester : MonoBehaviour  
{  
    public DialogueData testDialogue;  
  
    private void Start()  
    {  
        if (testDialogue != null)  
        {  
            Debug.Log($"=== 对话开始: {testDialogue.speakerName} ===");  
            foreach (string line in testDialogue.dialogueLines)  
            {  
                Debug.Log($"{{testDialogue.speakerName}}: {line}");  
            }  
            Debug.Log("=== 对话结束 ===");  
        }  
    }  
}
```

8.2 练习二：搭建简易对话系统

要求：实现一个包含 DialogueManager、DialogueUI、DialogueTrigger 的简易对话系统，支持按空格键推进对话。

系统架构：

组件	职责	依赖
DialogueData (ScriptableObject)	存储对话数据	无
DialogueManager (单例)	控制对话流程	DialogueData, DialogueUI
DialogueUI (MonoBehaviour)	显示对话内容	Unity UI 组件
DialogueTrigger (MonoBehaviour)	触发对话	DialogueManager, DialogueData

关键实现细节：

- DialogueManager 使用单例模式确保全局唯一
- 使用 isDialogueActive 标志位防止重复触发

- 使用协程实现打字机效果
- DialogueUI 通过 SerializeField 绑定具体的 UI 元素引用

8.3 练习三：分析 SKDialoguePlayer 及相关组件

要求：阅读 SKDialoguePlayer 源码，理解其架构设计和实现方式。

分析要点：

1. 状态管理：SKDialoguePlayer 使用了哪些状态来管理对话流程？每个状态对应什么行为？
 - Idle: 无对话，可接收新对话请求
 - Playing: 打字动画播放中，点击跳过动画
 - WaitingInput: 等待玩家确认继续
 - Choosing: 等待玩家选择选项
 - Finished: 对话结束清理
2. 事件系统：SKDialoguePlayer 定义了哪些事件？这些事件的订阅者分别是谁？
 - OnDialogueStart → 游戏输入系统禁用玩家移动
 - OnDialogueEnd → 游戏输入系统恢复玩家移动
 - OnNodeEnter → 音效系统播放对应音效
 - OnLineDisplayed → UI 更新当前文字
3. 节点解析：SKDialoguePlayer 如何从 ScriptableObject 数据构建运行时的对话树？
 - 使用 Dictionary<string, SKNodeData> 存储 ID 到节点的映射
 - 根据连接数据（SKConnectionData）构建节点间的引用关系
 - 启动时从 startNodeId 开始遍历
4. UI 解耦：SKDialoguePlayer 如何实现与 UI 的解耦？
 - 定义 IDialogueView 接口
 - SKDialoguePlayer 持有 IDialogueView 引用而非具体的 UI 类
 - UI 实现类通过依赖注入或 Inspector 赋值的方式传入

8.4 练习四：实现一个自定义对话节点

要求：扩展对话系统，实现一个“条件检查节点”，根据游戏变量决定对话分支。

解题思路：

1. 继承 DialogueNode 基类

2. 添加条件配置字段：条件键名、期望值、满足时跳转节点、不满足时跳转节点
3. 在 Execute 方法中从 DialogueContext 读取对应变量
4. 根据变量值与期望值的比较结果返回不同的后续节点

代码实现：

```
[CreateAssetMenu(menuName = "Dialogue/Nodes/Condition Node")]
public class DialogueConditionNode : DialogueNode
{
    public string variableKey;           // 变量键名
    public bool expectedValue;           // 期望值
    public DialogueNode trueNode;        // 条件为真时跳转
    public DialogueNode falseNode;      // 条件为假时跳转

    public override DialogueNode Execute(DialogueContext context)
    {
        if (context.flags.TryGetValue(variableKey, out bool value))
        {
            return value == expectedValue ? trueNode : falseNode;
        }
        // 变量不存在时默认走false分支
        return falseNode;
    }
}
```

9 总结与知识体系

9.1 本课核心知识点回顾

1. **ScriptableObject**: Unity 中数据持久化的核心工具，用于存储对话配置、角色信息等静态数据，支持编辑器内可视化编辑，内存高效（多引用共享单份数据）。
2. **简易对话系统架构**: 遵循数据-控制-视图分离原则，DialogueData（数据）→ DialogueManager（控制）→ DialogueUI（视图）的清晰分层。
3. **对话节点类型**:
 - 对话行节点（LineNode）: 显示一段文字
 - 选择节点（ChoiceNode）: 提供玩家选项
 - 分支节点（BranchNode）: 根据条件自动选择路径
 - 事件节点（EventNode）: 触发游戏逻辑
 - 条件节点（ConditionNode）: 运行时判断变量

4. UI 动画与互动:

- 打字机效果: 逐字显示, 标点符号有额外暂停
- 面板动画: 平滑的显示/隐藏过渡
- 头像切换: 淡入淡出过渡
- 选项交互: 悬停高亮、点击弹跳动画

5. 进阶对话系统:

- 可视化节点编辑器: 基于 EditorWindow 的自定义编辑界面
- 图数据结构: 节点和边的序列化存储
- 运行时解析: 从序列化数据构建可执行节点对象

6. SKDialoguePlayer:

- 状态机模式管理对话生命周期
- 观察者模式实现松耦合事件通知
- 命令模式封装节点行为
- 接口驱动设计实现 UI 解耦

9.2 对话系统在整个课程体系中的位置

对话系统是第四单元”玩法系统编程”的第一个模块, 后续还有:

- **L15 背包与物品系统:** 与对话系统经常联动 (NPC 通过对话给予物品、商人对话 + 商店 UI)
- **L16 战斗系统:** 战斗前后的对话、剧情推进
- **L17 归纳与前瞻:** 回顾 Unity 游戏开发核心技能, 展望 Shader 渲染、引擎工具开发

这些系统共享一套架构原则: ScriptableObject 管理配置数据、单例模式管理全局状态、事件系统实现模块间通信。掌握 L14 的对话系统设计思想, 将为后续学习打下坚实基础。

9.3 对话系统的进阶方向

对于有兴趣深入的同学, 可以进一步探索:

- **本地化支持:** 通过 ScriptableObject 存储多语言文本, 运行时根据语言设置加载对应文本
- **语音合成/配音:** 在对话节点上绑定 AudioClip, 实现角色配音
- **时间线系统:** 将对话与 TimeLine 结合, 实现过场动画式的对话演出
- **对话历史:** 保存已发生的对话记录, 支持回顾
- **Quest/任务系统集成:** 对话自动触发任务更新、导航引导
- **AI 驱动对话:** 结合大语言模型 (LLM) 实现动态生成的 NPC 对话